

SCIENCE INTERNSHIP PROGRAM

University of California, Santa Cruz

Science Internship Program (SIP) 2026

A Comparative Study of Classical and Semantic Communication Systems for BART Transit

Intern: Raj Sarbib

Mentor: Parisa Oftadeh

Summer 2026

Abstract

Reliable wireless communication is essential for real-time transit operations, where channel noise and signal attenuation can disrupt train coordination. This work compares a traditional classical communication system with a deep learning-based semantic communication model for transmitting Bay Area Rapid Transit (BART) telemetry for automated Hold-to-Transfer decisions. Both systems employ Hamming and Repetition channel coding with BPSK and 16-QAM modulation, while the semantic model first compresses the telemetry into a low-dimensional latent representation using a three-layer neural network before transmission. Both systems are evaluated under identical Additive White Gaussian Noise (AWGN) channel conditions across a range of signal-to-noise ratios (SNRs). Performance is measured using transfer decision accuracy, bit and symbol error rates, F1-score, transmission efficiency, and channel capacity. The semantic architecture reduces the telemetry payload from 256 bits to 12 bits (95%) while maintaining high decision accuracy under noisy conditions. Although the classical system achieves perfect accuracy at high SNR, the semantic model is more robust at low SNR, providing a better balance between communication efficiency and task performance. These results demonstrate the potential of semantic communication for bandwidth-constrained intelligent transportation and future railway communication systems.

Contents

1 Introduction **3**
- 1.1 Context and Motivation . . . 3
- 1.2 Research Objectives . . . 3
- 1.3 Related Works . . . 3

2 Problem Description **4**
- 2.1 System Model . . . 4
- 2.2 Data Pipeline and Input Formulation . . . 4
- 2.3 Channel Models . . . 5

3 Proposed Method **6**
- 3.1 Classical Communication System . . . 6
- 3.2 Semantic Communication System . . . 6

4 Comparison **7**
- 4.1 Classical - Transfer Hold Decision Accuracy . . . 7
- 4.2 Classical - System Bit Error Rate (BER) vs SNR . . . 8
- 4.3 Classical - Symbol Error Rate (SER) vs SNR . . . 8
- 4.4 Semantic - Transfer Hold Decision Accuracy . . . 9
- 4.5 Semantic - F1-Score . . . 9
- 4.6 Total Bits . . . 10
- 4.7 Capacity and Bandwidth . . . 10
- 4.8 Comparison of Power between Different Modulation Techniques . . . 11

5 Discussion **11**
- 5.1 Algorithmic Performance and Payload Optimization . . . 11
- 5.2 Hardware Efficiency and Directional Reliability . . . 12

6 Future Work **12**
- 6.1 GTFS Real-Time . . . 12
- 6.2 Hybrid Framework . . . 13

1 Introduction

1.1 Context and Motivation

The Bay Area Rapid Transit (BART) system generates a high volume of continuous telemetry data through its General Transit Feed Specification (GTFS) feeds [1]. This operational pipeline broadcasts everything from train positions to passenger loads. Transmitting these data payloads over weak channels or limited infrastructure presents significant problems when the information is essential for real-time public transportation coordination.

In underground transit networks, communications rely on wireless telemetry links, such as radio transceivers in tunnels, which are heavily affected by environmental noise and signal attenuation. For instance, at Embarcadero Station, the BART Operations Control Center (OCC) manages busy platform transfers by implementing an automated Hold-to-Transfer rule. If an incoming train's predicted delay is less than 180 seconds, the automated controller instructs a connecting train on the opposite side of the platform to keep its doors open. This helps passengers make their connections successfully.

Classical data transmission relies heavily on standard coding schemes and digital modulation, which can only mitigate small bit errors. When high channel noise causes bit flips, the automated controller makes critical dispatching mistakes. For westbound trains traveling through the Transbay Tube, failed transmissions due to tunnel noise lead to a Wrong Release, leaving East Bay commuters stranded in San Francisco. For eastbound trains traveling under Market Street, corrupted packets cause a Wrong Hold, forcing trains to wait unnecessarily at the platform and cascading delays throughout the entire transbay system [2].

Semantic communication introduces a foundational network that focuses more on context and downstream task completion than exact bit accuracy. By focusing only on the specific information needed to execute the transfer decision, this approach minimizes data overhead and ignores non-essential data components that do not affect operational performance.

1.2 Research Objectives

This project aims to develop and analyze two parallel telemetry communication pipelines, consisting of a traditional Classical System and a Deep Learning-based Semantic System. Both architectures are evaluated on their ability to accurately decide whether a BART train should be held at a connecting station given simulated wireless channel degradation.

The ultimate objective is to benchmark both architectures across a spectrum of Signal-to-Noise Ratios (SNR) measured in decibels. We evaluate them not only on absolute task accuracy to minimize Wrong Holds and Wrong Releases, but also on transmission efficiency and noise robustness.

1.3 Related Works

Traditional telemetry frameworks rely on a strict separation of source compression and channel coding, adhering to Shannon's classic separation theorem. In these architectures, physical-layer robustness against Additive White Gaussian Noise is achieved by balancing error-correcting code rates against digital modulation constellation densities.

In legacy transit telemetry configurations, error correction is often handled by block codes or repetition schemes paired with binary signaling. While a three-times Repetition code possesses higher nominal error correction capability per block, it introduces a 71 percent data overhead penalty compared to a standard Hamming (7,4) code, requiring 48 bits instead of 28 bits for a 16-bit payload. When total packet energy is fixed, spreading power across 48 bits lowers the energy

per coded bit, raising the pre-correction raw Bit Error Rate and rendering repetition less effective than Hamming in low-to-mid SNR ranges [3].

Modulation choice also dictates noise sensitivity. Binary Phase Shift Keying (BPSK) maps data onto two widely separated points on a real axis, maximizing geometric distance and noise tolerance. In contrast, 16-Quadrature Amplitude Modulation (QAM) packs 16 distinct symbols into a tight, two-dimensional complex grid. To normalize average transmission power to unit energy, the distance between neighboring 16-QAM points shrinks drastically, making it highly sensitive to minor noise displacements. Real-world engineering selects 16-QAM strictly to increase data throughput, not noise robustness.

2 Problem Description

2.1 System Model

The telemetry system treats a moving train approaching an underground platform as a discrete source that generates sequential telemetry updates. This telemetry data is transformed by an encoder, which can be configured as either a classical compression and channel-coding algorithm or a deep semantic feature extractor. The resulting symbols are exposed to a simulated noisy physical channel environment before reaching the receiver side. At the destination, a corresponding decoding architecture attempts to rebuild either the original operational parameters or the high-level semantic intent so they can be consumed by downstream transit dispatch applications.

2.2 Data Pipeline and Input Formulation

The input telemetry payload is composed of key information extracted from real-world transit entities combined with synthetically injected real-time operational variables. Real-world baseline coordinates, scheduled trip volumes, and route mappings are cross-referenced from official offline BART GTFS archives using the `stops`, `stop_times`, `trips`, and `routes` text files [1]. Real-time variables are mathematically modeled and injected into the code, including cumulative-sum station delays to represent sequential jitter, train speeds slowing to 35 miles per hour, compass bearings, and rush-hour passenger car loads ranging from 50 to 400 commuters [4].

The "Classical State Vector" is a one-dimensional NumPy array with 16 floating-point values. By putting all spatial, identity, and operational data into one memory block, a train's onboard computer can send its full status update in one compact message.
Array Index Layout and Definitions:

- **Indices 0, 1, 2, 3 (Historical Station Delays):** True historical timeline tracking how many seconds behind schedule the train was at its prior 4 stops leading up to the station.
- **Index 4 (Target Arrival Delay):** The definitive evaluation metric. This holds the projected delay for the upcoming Embarcadero platform. The automated controller parses this exact slot to compute the hold decision.
- **Indices 5 and 6 (Spatial Geolocation):** The fixed latitude and longitude coordinates from the Embarcadero GTFS station data.
- **Index 7 (Velocity):** The current operational speed of the train asset approaching the platform area.

- **Index 8 (Compass Bearings):** This shows the train's direction, set to 90.0° for Eastbound and 270.0° for Westbound.
- **Index 9 (Commuter Count):** This is the number of passengers on board, which shows how important it is to make the transfer connection.
- **Indices 10 to 15 (One-Hot Encoded Colors):** This is a binary block that shows the route color. Instead of sending text, each route color is mapped to a 6-element binary flag.

One-Hot Color Signature Mapping Matrix

When a train builds its packet, it sets six empty floating-point slots at indices 10 to 15 ([0.0, 0.0, 0.0, 0.0, 0.0, 0.0]). It then checks its GTFS line and sets the matching index to 1.0:

- **Red Line** → Array index 10 is activated: [1.0, 0.0, 0.0, 0.0, 0.0, 0.0]
- **Orange Line** → Array index 11 is activated: [0.0, 1.0, 0.0, 0.0, 0.0, 0.0]
- **Yellow Line** → Array index 12 is activated: [0.0, 0.0, 1.0, 0.0, 0.0, 0.0]
- **Green Line** → Array index 13 is activated: [0.0, 0.0, 0.0, 1.0, 0.0, 0.0]
- **Blue Line** → Array index 14 is activated: [0.0, 0.0, 0.0, 0.0, 1.0, 0.0]
- **Other/Special Lines** → Array index 15 is activated: [0.0, 0.0, 0.0, 0.0, 0.0, 1.0]

The mixed nature of this vector requires an architecture capable of processing continuous spatial numbers, temporal integers, and categorical text strings simultaneously. The classical system must encode this entire 16-element block precisely to make a downstream decision. Conversely, the semantic system aims to map this vector directly to its task-oriented operational meaning, ensuring robust Hold-to-Transfer execution even when individual indices are severely corrupted by the communication channel [5].

2.3 Channel Models

The physical layer link budget for both competing architectures is modeled using Additive White Gaussian Noise (AWGN) to simulate the signal attenuation encountered in underground rapid transit. To ensure a fair evaluation, both the classical and semantic pipelines share an identical channel infrastructure. This shared baseline guarantees that any variance in downstream dispatch control performance stems entirely from coding paradigm efficiency rather than unequal noise injection.

The mathematical formulation transitions the SNR from a decibel scale to a linear power ratio to isolate the required noise variance. The system evaluates a broad operating envelope ranging from minus ten decibels to ten decibels, adjusting the noise floor dynamically based on the selected digital modulation scheme. BPSK enforces a normalized average symbol energy of 1.0 and a data density of 1 bit per symbol. In contrast, the high-density 16 QAM grid scales its raw constellation coordinates to maintain an average symbol power of ten point zero while packing four bits per symbol [3].

The noise standard deviation, denoted as sigma, is calculated by taking the square root of the average symbol energy divided by the quantity of two point zero multiplied by the bits per symbol

and the linear scale SNR. The noise contribution is split evenly across a two-dimensional complex plane to account for phase and amplitude distortions. The real axis receives an in-phase noise component generated from a normal distribution scaled by sigma, while the imaginary axis receives a quadrature noise component generated from a matching independent normal distribution. The received signal vector is finally established as the complex sum of the transmitted symbols and the combined noise vectors, forcing the demodulator to resolve corrupted symbols along the complex coordinate plane [3].

3 Proposed Method

3.1 Classical Communication System

The classical pipeline adheres strictly to a source-channel coding architecture, treating every bit in the sixteen-element state vector with equal importance regardless of its operational utility. When a train updates its status, the continuous and categorical variables are compiled and digitized into a raw sequence of bits. This data block is passed into a forward error correction engine that can be configured to use either a Hamming or a Repetition block coding routine.

Once the channel coding is complete, the expanded bit sequences are mapped to complex waveforms. For Binary Phase Shift Keying, logic ones map to positive one point zero and logic zeros map to negative one point zero along the real axis line. For the sixteen-quadrature amplitude modulation grid, the bits are clustered into four-bit nibbles and mapped directly onto a gray-coded complex constellation containing sixteen closely packed coordinate points. At the receiver side, a maximum-likelihood detector evaluates the minimum Euclidean distance between the noisy incoming vector and the valid constellation points to yield demodulated bits. Finally, a decoding syndrome check applies parity matrix calculations to correct isolated single-bit errors or flags a decoding failure if uncorrectable bit corruption shifts the underlying delay parameters.

3.2 Semantic Communication System

The semantic pipeline replaces the separate source compression, channel coding, and modulation steps with a cohesive deep learning architecture. This approach utilizes a PyTorch neural network called the Semantic Feature Extractor Network, which is designed to identify and isolate the time-sensitive delay vectors while compressing non-essential spatial data components. The network is trained end-to-end for 250 epochs to ensure that the model pulls the best information possible. The architecture consists of two interconnected heads: a deep structural encoder and a matching downstream task decoder [5].

The encoder block takes the raw sixteen-element state vector directly as an input. It processes this data through three sequential linear layers. The first layer projects the sixteen dimensions down to twelve nodes. The second hidden layer compresses the twelve dimensions into eight nodes, and the final layer condenses these variables down into a continuous three-dimensional bottleneck, which represents the essential semantic features. To transmit this latent vector across the digital channel, a quantization routine clips the continuous features between a bound of minus three point zero and positive three point zero. It scales these values into a baseline twelve-bit digital source packet, which means that the semantic system requires only twelve total bits compared to the classical baseline.

The twelve quantized bits are modulated (BPSK and QAM 16), passed through the complex noise channel model, and demodulated back into binary form. The receiver then runs a dequantization routine to reconstruct the three-dimensional vector. This vector is fed into the decoder

head of the neural network, which consists of three linear layers mapping from three features up to eight nodes, then up to twelve nodes, and finally to a single output neuron. A sigmoid activation function restricts this output layer to a continuous probability between 0.0 and 1.0. The automated dispatcher compares this value against a 0.5 threshold, executing a flawless hold decision even when the underlying data bits have experienced severe channel corruption [3].

4 Comparison

4.1 Classical - Transfer Hold Decision Accuracy

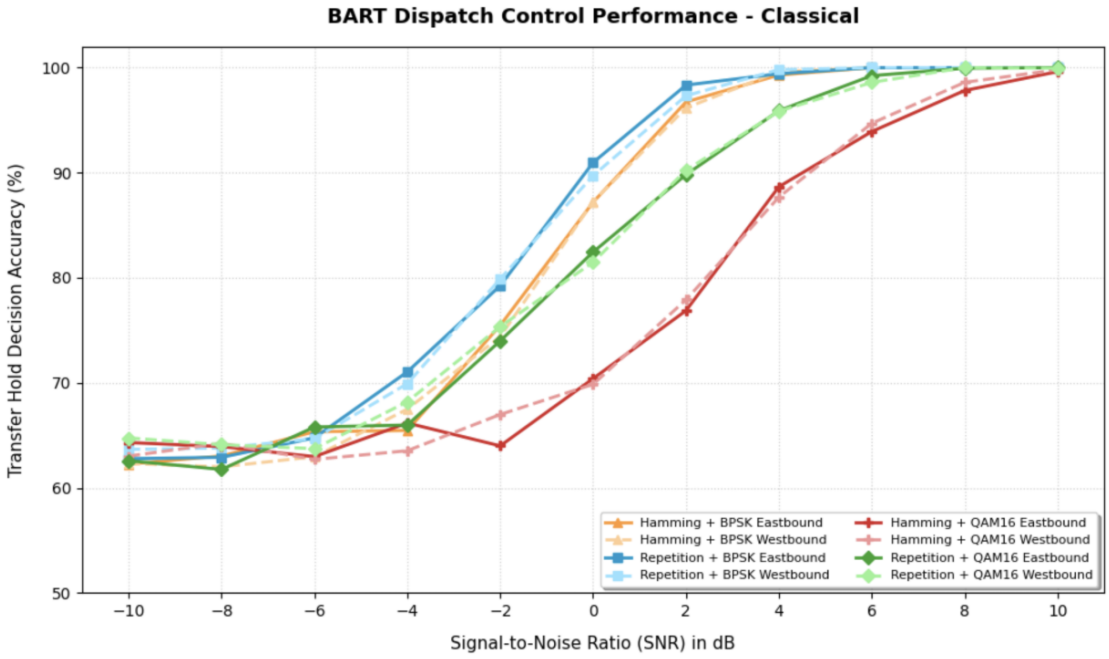


Figure 1: BART Dispatch Control Performance - Classical

The accuracy for all four combinations improves as the SNR increases; however, they do so at different rates. Looking at the graph, BPSK outperforms QAM 16, and Repetition does better than Hamming. All data eventually converge to around 100% at 10 SNR.

4.2 Classical - System Bit Error Rate (BER) vs SNR

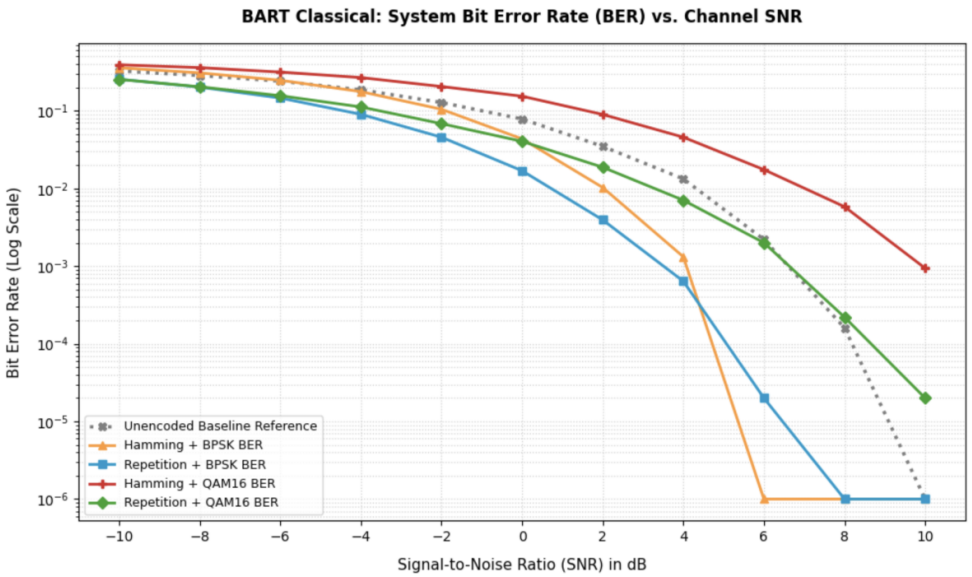


Figure 2: Classical - System Bit Error Rate (BER) vs SNR

We see that the error rate drops steeply as SNR increases. The unencoded baseline, which is simply BPSK with no coding scheme, performs better than the Hamming + QAM 16. Again, we see that BPSK outperforms QAM 16, as the blue and yellow perform the best.

4.3 Classical - Symbol Error Rate (SER) vs SNR

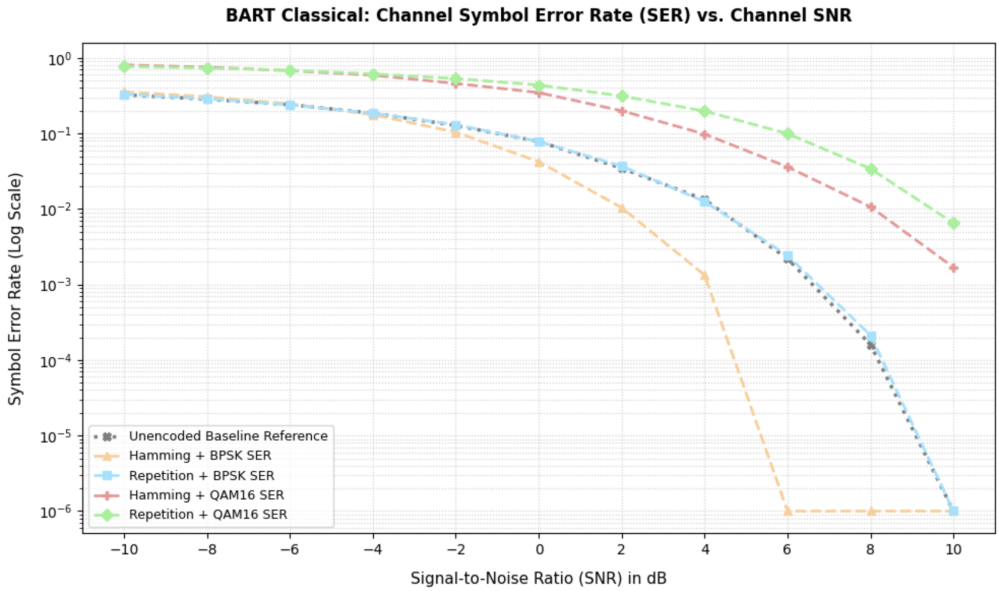


Figure 3: Classical - Symbol Error Rate (SER) vs SNR

The frequency of symbol errors steadily diminishes for each combination when channel conditions improve, falling at varying paces. Based on the plot, BPSK maintains a much lower error rate than QAM 16, and Repetition proves more resilient than Hamming. All lines ultimately plunge down toward a microscopic error floor as the signal hits 10 SNR.

4.4 Semantic - Transfer Hold Decision Accuracy

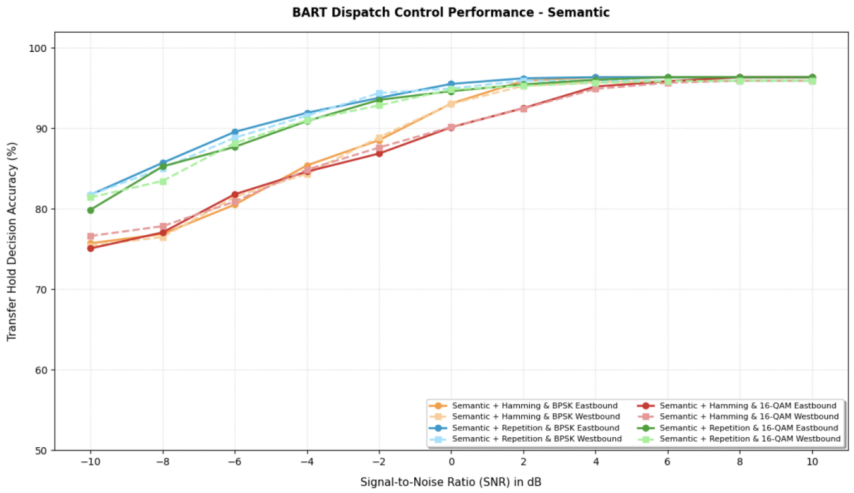


Figure 4: Semantic - Transfer Hold Decision Accuracy

As the SNR grows stronger, decision accuracy across all four setups increases, though at noticeably distinct speeds. Looking at the trends, BPSK consistently beats QAM 16, while Repetition coding holds an edge over Hamming. Every configuration eventually climbs and levels off near 96% accuracy by the time it reaches 10 SNR.

4.5 Semantic - F1-Score

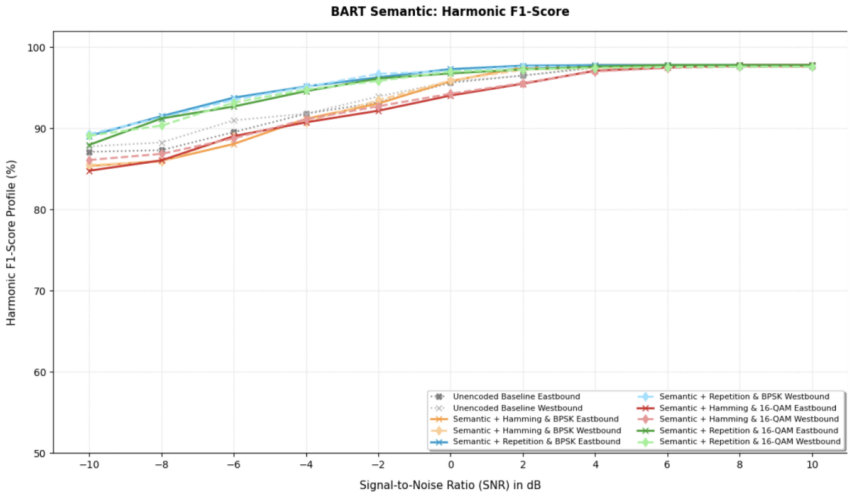


Figure 5: Semantic - F1-Score

The F1-score serves as the harmonic mean of precision and recall to evaluate how accurately your semantic machine learning model classifies transit decisions without being biased by unbalanced data.

The mathematical equations behind this metric are structured as follows:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}} \tag{1}$$

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}} \tag{2}$$

$$\text{F1} = 2 \times \left(\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right) \tag{3}$$

Precision measures the proportion of positive predictions that are actually correct, while recall measures the percentage of actual positive cases that the model successfully identifies.

Every combination shows a steady rise in its F1-score as the signal-to-noise ratio climbs, with each curve advancing along its own path. The graph clearly shows BPSK leading ahead of QAM 16, alongside Repetition outperforming the Hamming setups. Performance across the board finishes by merging together right around 96% at 10 SNR.

4.6 Total Bits

	Classical	**Semantic**
Raw-Bits	256	12
Hamming	448	21
Repetition	768	36

Table 1: Total Bits Comparison Matrix

4.7 Capacity and Bandwidth

BART Channel Bandwidth (B): 12500 Hz

The Capacity column in your table is calculated using the Shannon-Hartley Theorem, which determines the absolute maximum error-free data rate of a channel based on its bandwidth and noise level.

The channel capacity equation is defined as:

$$C = B \times \log_2(1 + \text{SNR}_{\text{Linear}}) \tag{4}$$

The resulting airtime numbers in milliseconds are derived by dividing the total payload bits by this capacity value:

$$\text{Airtime (ms)} = 1000 \times \left(\frac{\text{Total Bits}}{\text{Capacity (bps)}} \right) \tag{5}$$

SNR (dB)	Capacity C (kbps)	Classical Airtime (ms)	Semantic Airtime (ms)
-10	1.72	148.942	6.982
-4	6.04	42.360	1.986
0	12.50	20.480	0.960
4	22.65	11.301	0.530
10	43.24	5.920	0.278

Table 2: Channel Capacity and Airtime Metrics across SNR Levels

- **Semantic Data Compression Advantage:** 95.31% fewer bits required.

4.8 Comparison of Power between Different Modulation Techniques

PAPR measures the efficiency of your wireless transmitter power amplifier by tracking the relationship between peak signal power and average signal power.

The mathematical relationships for PAPR are calculated using these equations:

$$\text{PAPR} = \frac{\text{Peak Power}}{\text{Average Power}} \tag{6}$$

$$\text{PAPR (dB)} = 10 \times \log_{10}(\text{PAPR}) \tag{7}$$

	BPSK	QAM 16
Average Power	1.00	1.00
Peak Power	1.00	1.80
Peak to Average Power Ratio (PAPR)	1.00 (0 dB)	1.80 (2.55 dB)

Table 3: Power Performance Metrics for Binary Phase Shift Keying and 16-QAM

5 Discussion

5.1 Algorithmic Performance and Payload Optimization

The role of channel coding highlights a divergence between classical and semantic communication. In classical communication, channel coding schemes like Hamming or Repetition are absolutely critical. Because classical architectures treat every raw bit with equal weight, a single uncorrected bit inversion can corrupt an entire data set. When noise levels rise, classical systems have no choice but to send more, increasing the raw 256-bit packet to 768 bits under Repetition coding. This reliance creates a trade-off: to maintain data integrity in low SNR environments, the classical framework must sacrifice massive amounts of channel capacity and airtime, though it eventually rewards the system with perfect 100% accuracy once the channel clears.

Semantic communication reduces this dependence on error correction by embedding resilience into the data itself. Because the neural network compresses information into a low-dimensional semantic space, the system transmits data characteristics rather than bit arrays. If noise corrupts a portion of a semantic packet during transmission, the deep learning model at the receiver can still infer the intended message by analyzing the remaining, uncorrupted feature space. We see this in the accuracy, as all schemes start around 75-80% even at a low -10 SNR, where the classical

starts at 60-65%. Consequently, while transitioning from a 12-bit raw semantic packet to a 36-bit Repetition-coded packet does provide a helpful stability boost in extreme noise, it is not a strict requirement for basic system functionality. The underlying machine learning model acts as its own implicit layer of error correction, allowing the system to maintain a high F1-score without relying on bandwidth-heavy coding [6].

However, semantic communication has a ceiling of roughly 96% accuracy that cannot be overcome by simply improving channel conditions or adding stronger coding. In a classical framework, a perfect 10 dB SNR channel allows error correction to deliver 100% performance because the raw data bits remain uncompressed. The semantic system, by contrast, relies on a lossy encoder that permanently discards fine-grained data details at the source to achieve its tiny payload size. Even if the wireless channel is completely free of noise, the receiver can only reconstruct the message based on the limited latent features it receives. Ultimately, channel coding serves as an essential fix for classical vulnerabilities, whereas semantic systems trade away that final 4% of absolute perfection to achieve a permanent 95% reduction in packet size and a 21x speedup in airtime.

5.2 Hardware Efficiency and Directional Reliability

The Symbol Error Rate graph reveals that 16-QAM struggles in heavy noise because its signaling points are packed tightly together, whereas BPSK keeps errors much lower by using widely separated signals. More importantly, the Harmonic F1-Score graph tracks both Eastbound and Westbound trip paths, and the data lines for both directions overlap almost perfectly. This symmetry proves that your semantic model is stable and handles different travel trajectories reliably without any directional bias. Furthermore, the F1-score curves flatten completely after crossing 4 dB SNR. This flat line represents a clear point of diminishing returns, meaning that once your signal crosses 4 dB, extra signal strength yields no performance benefits and you can safely lower transmitter power to save resources.

The Peak-to-Average Power Ratio numbers from your code back up this hardware advantage. BPSK has a perfect power ratio score of 1.00 because its signal wave is completely flat and never spikes. On the other hand, 16-QAM has a fluctuating power ratio of 1.80. High power spikes are a major problem for physical engineering because they force wireless transmitters to use expensive, power-hungry linear amplifiers to avoid signal distortion [7]. Because the BPSK setups have zero power spikes, they run with maximum amplifier efficiency. Choosing a BPSK configuration alongside your semantic model does more than just lower error rates over the air; it physically protects your edge transmitter devices from draining their batteries and overheating.

6 Future Work

6.1 GTFS Real-Time

To enhance the real-world utility of the current system, I would transition from GTFS timetables to GTFS Real-Time data streams [1]. Incorporating real-time feeds will allow the system to evaluate model adaptability against unexpected transit delays passenger density spikes across the BART network. Currently, these numbers are generated for both classical and semantic systems. Testing under these unpredictable cases will ensure that the semantic feature space remains resilient when mapping vehicle positions and scheduling communication windows under actual operating conditions.

6.2 Hybrid Framework

Building upon this foundation, I would also work on creating an adaptive hybrid communication framework to optimize the system's performance boundaries. This dual-mode architecture will dynamically shift between classical and semantic frameworks based on real-time channel feedback. When the wireless channel experiences clear conditions with a high signal-to-noise ratio (SNR), the pipeline will trigger classical transmission to bypass the 96% semantic accuracy ceiling and achieve perfect 100% data reconstruction [2]. Conversely, during periods of heavy signal noise, the system will fall back to semantic mode to maintain robust link stability, while safely throttling transmitter power once the channel crosses the 4 dB efficiency threshold to conserve trackside resources.

References

[1] "BART developer data: General transit feed specification (GTFS)," https://bart.gov, 2026, accessed 3 Aug. 2026.

[2] F. Hu, Z. Ling, T. Liu, H. Li, and B. Ai, "Wireless perception of high-speed railway communication: Challenges, framework, and future directions," *IEEE Wireless Communications*, vol. 31, no. 4, pp. 284–292, August 2024.

[3] E. Krouk and S. Semenov, *Modulation and Coding Techniques in Wireless Communications*. Wiley, 2011, google Books. [Online]. Available: https://google.com

[4] D. Liu *et al.*, "Analyzing transit systems using general transit feed specification (gtfs) by generating spatiotemporal transit networks," *Information*, vol. 16, no. 1, p. 24, 2025, accessed 3 Aug. 2026. [Online]. Available: https://doi.org

[5] S. Kadam and D. I. Kim, "Semantic communication-empowered vehicle count prediction for traffic management," *arXiv preprint arXiv:2307.12254*, 2023, accessed 3 Aug. 2026. [Online]. Available: https://arxiv.org

[6] I. Floudas, M. Anastasopoulos, A. Tzanakaki, and J. G. Terán, "Experimental evaluation of semantic communications for 6G networks in railway systems," *IEEE Communications Standards Magazine*, vol. 9, no. 4, pp. 34–40, Dec. 2025.

[7] D. Wulich, L. Goldfeld, and G. R. Tsouri, "Peak to average power ratio in digital communications," in *Proceedings of Ben-Gurion University of the Negev (Beer-Sheva 84105)*, 2005, pp. 779–782.